GEARBOX / INTEGRATION BRIEF

Connecting Django to SAP S/4HANA

Sandbox test plan, connector assessment and integration blueprint

Prepared for engineering and ERP integration stakeholders

13 November 2026 · Version 1.0

SCOPE NOTE

Architecture-level assessment. The private erp-connector repository was not accessible during this review. Connector claims are marked as reported, not code-verified. No client data. Not SAP-certified.

BusDec
github.com/BusDec

Contents

1. Executive answer . . . 3
2. Scope, evidence and how to read this guide . . . 4
3. Current state: what each piece actually provides . . . 5
4. Target architecture . . . 6
5. SAP integration surfaces . . . 7
6. Stage 1: free sandbox test (read-only) . . . 8
7. Stage 2: write-path testing on a real tenant . . . 10
8. Canonical model and field mapping . . . 11
9. Django implementation blueprint . . . 12
10. Sync, error handling and operations . . . 14
11. Security and compliance . . . 16
12. Test strategy and acceptance criteria . . . 16
13. Delivery plan and responsibilities . . . 17
14. Risks, open questions and next steps . . . 18
Appendix A. API inventory and request cookbook . . . 19
Appendix B. Glossary and references . . . 20

How to use this guide
Decision makers: read sections 1-3 for the verdict, the evidence and the gaps. **Architects and developers:** sections 4-9 cover the target design, SAP surfaces, the two test stages, mapping and the Django blueprint. **Delivery and operations:** sections 10-14 cover failure handling, security, testing, the plan and the risks. The appendices are a field checklist and a glossary.

1. Executive answer

The two repositories are **not enough** to connect the Django application to SAP. They are the right starting point, but they cover different layers of the problem.

3	1	0
code assets in the story	live SAP endpoint checked	write operations testable in the free sandbox

Asset	What it is	Enough on its own?	What is still missing
cmms-demo	Public Next.js static showcase with synthetic data.	No	Backend, database, authentication and any live ERP call.
erp-connector	Private SAP S/4HANA adapter, simulator and contract tests (reported).	No, but necessary	Django persistence, scheduling, deployment and UAT.
Django app	The application that owns users, canonical data and the operating UI.	Required	Not present in this workspace; needs the integration layer.
SAP sandbox	Free Business Accelerator Hub sandbox.	Only for Stage 1	Read-only: no POST/PATCH, no business roles, no SLA.
SAP test tenant	A real S/4HANA test system with a communication arrangement.	Required for Stage 2	Provisioning, credentials, EAM scope and test master data.

The short answer
Use **cmms-demo** as the canonical-model and UX reference. Use **erp-connector** as the SAP transport and mapping layer. Build the Django persistence, sync and operations layer. Then test in two stages: a free read-only sandbox first, and a real S/4HANA test tenant for writes. The sandbox proves the client, authentication header, paging and mapping; it cannot prove a maintenance order can be created.

What can be proven this week

- **Read path:** the sandbox can return equipment, functional locations, notifications and orders for real SAP API contracts.
- **Field mapping:** map a sample of each entity into the canonical model and record every unmapped or ambiguous field.
- **Client behaviour:** pagination, OData filters, JSON shape, error handling and rate-limit behaviour can be exercised.
- **Deployment shape:** the Django app can store and display canonical records without any SAP write permission.

What cannot be proven in the free sandbox

- **Writes:** creating notifications, orders or confirmations; changing status; posting labour.
- **Authorisation:** the business role, communication user and plant-level restrictions.
- **Business validation:** order types, planner groups, work centres, bill of material and costing rules.
- **Throughput:** the sandbox is not a performance or volume test environment.

Minimum path to a working pilot

Step	Outcome	Typical effort	Owner
0. Access	Connector and Django code reachable; SAP account and API key available.	1-2 days	You + engineering
1. Sandbox	Four read-only APIs return data; mapping notes signed off.	3-5 days	Django developer
2. Persist	Canonical tables, sync runs, external IDs and admin screens.	5-8 days	Django developer
3. Tenant	Communication arrangement, OAuth2 token, write E2E for one scenario.	5-10 days	SAP basis + engineering
4. Harden	Retries, reconciliation, monitoring, secrets and runbook.	3-5 days	Engineering + ops
5. UAT	Breakdown notification -> order -> confirmation accepted by maintenance.	3-5 days	Maintenance SME + QA

The critical path is usually not code. It is SAP access: a test tenant, a communication arrangement and a business role that actually contains the EAM APIs. Start that request on day one.

2. Scope, evidence and how to read this guide

This is an architecture and delivery guide, not a claim that the integration is production-ready. Every statement is labelled by its evidence level so a reader can tell what was inspected, what was reported by the product team, and what still needs confirmation in SAP.

Label	Meaning	Examples in this guide
VERIFIED	Checked directly against the repository, a live endpoint or public documentation.	The demo is a static export; the sandbox returns 401 without a key.
REPORTED	Stated by the product material but not code-reviewed here.	The connector supports OAuth2, paging, retries and mapping.
TO CONFIRM	Must be checked in the tenant, the connector code or the Django codebase.	Entity-set names, EAM API availability, scopes and field maps.

Important limitation
The private **erp-connector** repository returned 404 to unauthenticated access, and no Django application is present in this workspace. The connector sections therefore describe the expected contract and the questions to ask of the code. They do not claim a line-level review. Once the code is available, this guide should be updated with the actual module names, supported OData versions and implemented API coverage.

Document control

Item	Value
Version	1.0
Date	13 November 2026
Prepared for	Engineering lead, Django developer, ERP integration stakeholder
Scope	SAP S/4HANA read-only sandbox, Django integration blueprint, repository sufficiency
Out of scope	SAP ECC, IBM Maximo and Oracle EAM implementation; production cutover; SAP certification
Evidence base	cmms-demo source and product notes; live DNS/TLS and sandbox endpoint checks; SAP public documentation
Limitation	erp-connector and the Django application were not available for code review

3. Current state: what each piece actually provides

The Gearbox story is deliberately honest: the demo is synthetic, the connector is private, and the other ERP adapters are planned. That separation is useful, but it means no single repository is an end-to-end SAP integration.

3.1 cmms-demo

- **VERIFIED** — Next.js 15 App Router with `output: 'export'`; the build produces static files in `out/`.
- **VERIFIED** — All plant data is generated deterministically in `lib/data.ts`; there is no database and no live ERP call.
- **VERIFIED** — The canonical model page describes functional locations, equipment, notifications, work orders, operations, materials, confirmations and meters.
- **VERIFIED** — The ERP integration page lists the SAP APIs and labels the S/4HANA adapter as the reference adapter, with ECC, Maximo and Oracle as planned.
- **Use it for:** the canonical vocabulary, field ownership, dashboard expectations and the demo narrative. **Do not use it as** the integration runtime.

3.2 erp-connector

- **REPORTED** — Contains the S/4HANA adapter, OData client, OAuth2 flow, pagination, retry policy, canonical mapping, a local simulator and CI contract tests.
- **REPORTED** — Described publicly as a standalone library, not a copy of any one CMMS.
- **TO CONFIRM** — Which APIs are implemented, whether V2 and V4 are both supported, how errors are normalised, whether writes are implemented, and how configuration/secrets are injected.
- **Use it for:** transport, authentication, paging and mapping. **Add** the Django service, persistence and operations layer around it.

3.3 The Django application

No Django project is present in `/Users/bnb/Documents/Projects/cmms-demo`. The Django app is the missing consumer: it owns users, permissions, canonical tables, sync state, the internal API and the operating UI. The guide assumes a standard Django + PostgreSQL deployment with a task runner; adjust the names to the real project.

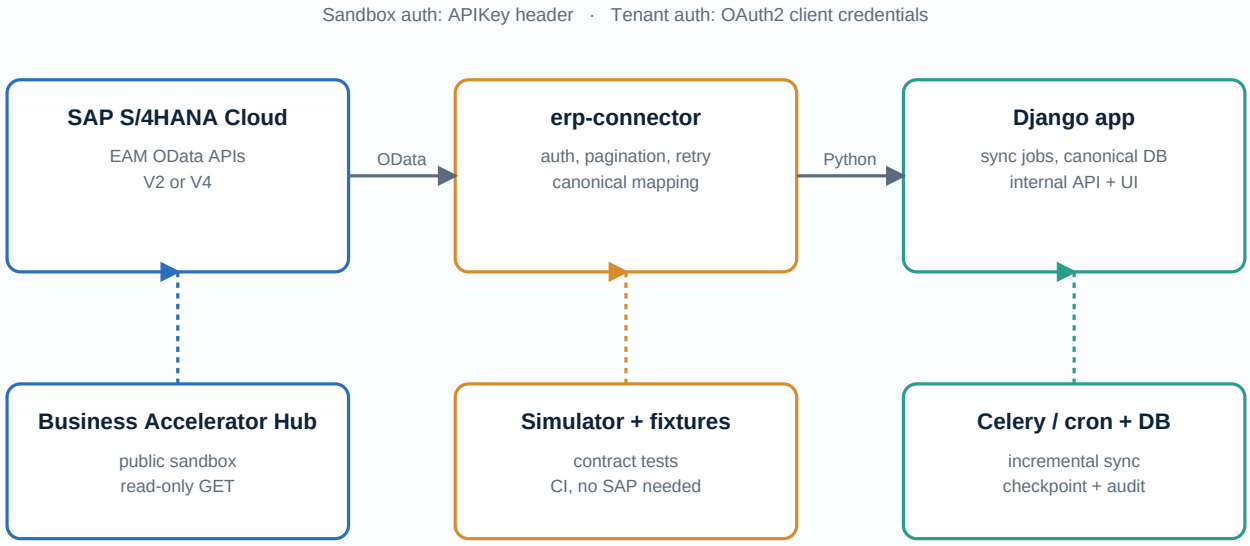
3.4 SAP landscape

Environment	Auth	Writes	Data	Use in this plan
Business Accelerator Hub sandbox	APIKey header	No (GET only)	Sample, resets	Stage 1: client, paging, mapping
S/4HANA Cloud trial	OAuth2 / communication arrangement	Limited by scope	Trial tenant	Stage 2 if EAM APIs are included
Customer test / sandbox tenant	OAuth2 client credentials	Yes, with role	Test client data	Stage 2 and UAT (recommended)
Production	OAuth2, least privilege	Yes	Live	Only after UAT and change approval

The public sandbox is excellent for de-risking the read path before anyone provisions a tenant. It is not a substitute for the tenant when the question is “can we create a notification and an order?”.

4. Target architecture

The clean separation is: SAP remains the system of record; the connector owns SAP-specific transport and mapping; Django owns the canonical model, users, workflow and sync state. The demo's data-model page already states the same principle: adapters map each source into the same canonical entities and the reporting layer never reads SAP tables directly.



The connector is the only component that knows SAP field names. Django stores canonical records and sync state.

4.1 Library or separate service?

Option	How it works	Strengths	Trade-offs	Recommendation
A. Library inside Django	Django imports the connector package and calls it in a service layer.	Fewest moving parts; fastest to pilot; one deployment.	SAP calls share the web/worker process; coupling to the connector release.	Start here for the pilot.
B. Sidecar service	A small internal HTTP service wraps the connector; Django calls it.	Isolation, independent scaling, reusable by other apps.	Another deployable, auth between services, more ops.	Move here if multiple consumers or strict network isolation.
C. Batch CLI only	A scheduled job pulls SAP data into Django; no synchronous calls.	Simple, robust for nightly snapshots.	Higher latency; no near-real-time status.	Use as a fallback for large master-data loads.

The public description calls the connector a standalone library, so option A is the lowest-friction fit. Keep the connector boundary explicit: Django services should not contain OData URLs, SAP field names or token logic.

4.2 Data flow

- **Extract:** the connector reads a page from an OData entity set, normalises V2/V4 JSON and returns rows plus a next cursor.
- **Bronze:** optionally persist the raw payload and a content hash for replay and debugging.
- **Silver:** map to canonical entities, deduplicate by external ID, map codes and units, validate required fields.
- **Gold:** compute KPI snapshots or expose canonical tables to the dashboard and BI layer.
- **Checkpoint:** commit the page and the cursor in one transaction; never advance the cursor before the rows are stored.

5. SAP integration surfaces

The Gearbox demo lists the SAP API inventory. Treat that list as a starting point, not a contract: confirm the exact service, version, entity set and released scope in the tenant. The API names below are the ones used by the demo and the public SAP catalogue.

Canonical entity	SAP API	Typical entity set	Direction	Purpose
Functional location	API_FUNCTIONALLOCATION	FunctionalLocation	Inbound	Location hierarchy and plant assignment
Equipment	API_EQUIPMENT	Equipment	Inbound	Asset master, category, manufacturer, serial
Maintenance notification	API_MAINTNOTIFICATION	MaintenanceNotification	Bidirectional	Request, breakdown, priority and dates
Maintenance order	API_MAINTENANCEORDER	MaintenanceOrder	Bidirectional	Order header, status, dates and cost
Order operation	API_MAINTENANCEORDER	MaintenanceOrderOperation	Bidirectional	Work centre, planned/actual work
Confirmation	API_MAINTENANCEORDERCONFIRMATION	MaintenanceOrderConfirmation	Outbound	Labour, time and final confirmation
Measuring point	API_MEASURINGPOINT	MeasuringPoint	Inbound	Meters, units and condition readings
Material / spare	API_MAINTENANCEORDER (or material API)	To confirm	Inbound	Reserved or consumed components

Entity-set names and available navigation properties must be read from each service's `$metadata`. Do not hard-code them from a slide; the catalogue can expose more than one service version for the same business object.

5.1 Sandbox vs production authentication

Concern	Business Accelerator Hub sandbox	S/4HANA tenant
Base URL	https://sandbox.api.sap.com/s4hanacloud/sap/opu/odata/sap/<SERVICE>/	https://<tenant-host>/sap/opu/odata/sap/<SERVICE>/
Credential	Personal API key in the APIKey header	OAuth2 client credentials from a communication arrangement
Token	None	Bearer token with expiry; cache and refresh
Writes	Not supported (GET only)	Supported within the communication user's role and scope
Data	Sample data; resets periodically	Client test data; change control applies
SLA	None	Per the SAP contract

Do not build production auth from the sandbox
The APIKey header is a sandbox convenience. A production S/4HANA integration authenticates with OAuth2 through a communication arrangement (communication system, communication user, business role and scopes). Design the connector so the auth strategy is an injectable provider, not a hard-coded header.

5.2 What the sandbox check confirmed

On 13 November 2026, unauthenticated requests to the sandbox metadata endpoints for API_MAINTENANCEORDER, API_EQUIPMENT, API_MAINTNOTIFICATION and API_FUNCTIONALLOCATION returned HTTP 401. That confirms the base URL pattern and that a credential is required. It does not confirm the entity sets, the data or the permissions of a specific API key.

```
BASE = https://sandbox.api.sap.com/s4hanacloud/sap/opu/odata/sap
BASE/API_MAINTENANCEORDER/$metadata  -> 401 without APIKey
BASE/API_EQUIPMENT/$metadata         -> 401 without APIKey
```

6. Stage 1: free sandbox test (read-only)

Objective: prove the connector's read path and the canonical mapping against real SAP API contracts, before requesting a tenant or writing any Django persistence. This stage should take three to five working days for one developer.

6.1 Get access

- Create a free account on the **SAP Business Accelerator Hub** (api.sap.com). Use a company email; the account is personal.
- Search for the four EAM APIs: API_EQUIPMENT, API_FUNCTIONALLOCATION, API_MAINTNOTIFICATION and API_MAINTENANCEORDER.
- Open each API and read the overview, API reference and the 'Try it out' console. Download the EDMX/OpenAPI specification if the page offers it.
- Open your profile menu and copy the personal **API key**. Treat it as a secret: do not commit it, paste it into a ticket or put it in a browser URL.
- Set it as an environment variable, for example `SAP_API_KEY`, in the shell or secret manager used for the test.

6.2 Smoke test the endpoint

```
export SAP_API_KEY="<your personal key>"
BASE="https://sandbox.api.sap.com/s4hanacloud/sap/opu/odata/sap"
curl -s -H "APIKey: $SAP_API_KEY" \
  "$BASE/API_EQUIPMENT/\$metadata" | head -n 20
```

If the metadata returns XML, the key and base URL are correct. If it returns 401, regenerate the key. If it returns 403, the key is valid but the API is not enabled for that account. If it returns 404, the service name or path is wrong.

6.3 Query one page

```
curl -s -H "APIKey: $SAP_API_KEY" -H "Accept: application/json" \
  "$BASE/API_EQUIPMENT/Equipment?\$top=5&\$select=Equipment,EquipmentName,EquipmentCategory"
```

Entity-set names are service-specific. If `Equipment` returns 404, search the `$metadata` for `EntitySet` entries and use the exact name. The same rule applies to notifications and orders.

6.4 A minimal Python client

```
import os
import requests

BASE = "https://sandbox.api.sap.com/s4hanacloud/sap/opu/odata/sap"
HEADERS = {
    "APIKey": os.environ["SAP_API_KEY"],
    "Accept": "application/json",
}

def get_page(service, entity_set, top=50, cursor=None):
    url = f"{BASE}/{service}/{entity_set}"
    params = {"$top": top}
    if cursor:
        params["$skiptoken"] = cursor
    response = requests.get(url, headers=HEADERS, params=params, timeout=30)
    response.raise_for_status()
    payload = response.json()
    # OData V2 wraps rows in d.results; V4 uses value.
    rows = payload.get("d", {}).get("results", payload.get("value", []))
    next_link = payload.get("d", {}).get("__next") or payload.get("@odata.nextLink")
    return rows, next_link
```

The connector should already normalise the V2/V4 difference. If it does not, add a thin adapter in Django rather than letting the difference leak into every mapper.

6.5 What to test

Test	What good looks like	Record
Metadata	HTTP 200, XML with EntitySet and Property definitions.	Service name, version, entity sets.
Single page	Rows parse, required fields present, types are stable.	Sample IDs and field names.
Pagination	Next link/cursor is followed; no duplicate or missing rows at page boundaries.	Page size, cursor field, total count.
Filter / select	Only the requested fields return; date and status filters behave as expected.	Supported operators and date format.
Errors	401, 403, 404, 400, 429 and 5xx are distinguishable.	Status code, SAP message, retry guidance.
Mapping	10 equipment, 5 locations, 5 notifications, 5 orders map to canonical fields.	Unmapped fields and decisions.

6.6 Sandbox limits

- **Read-only:** only GET is supported. Do not build a write test around the public sandbox.
- **No SLA:** it can be slow or unavailable; do not put it in a production CI gate.
- **Resets:** sample data can change; record the IDs you used and do not depend on them long-term.
- **Personal key:** the key identifies your account; do not share it or embed it in a demo build.
- **Not a tenant:** no business roles, communication arrangement, plant restrictions or customising.

> **Stage 1 exit criteria**
> Four EAM APIs return metadata and at least one page; pagination is proven on one API; the core canonical mappings have a written decision for every unmapped field; errors are classified; no secret is in the repository; the connector's read path is exercised through its real interface, not a copy-pasted requests script.

7. Stage 2: write-path testing on a real tenant

The public sandbox cannot validate writes. To prove that Django can create a notification, create an order and post a confirmation, you need an SAP S/4HANA test system with a communication arrangement and a business role that contains the EAM APIs. This is the step that most often delays a pilot, so start the access request in parallel with Stage 1.

7.1 Choose the tenant

Option	Fit	Watch-outs
Customer test / sandbox tenant	Best: real customising, roles, plant data and write behaviour.	Change control, transport requests, test-data ownership.
S/4HANA Cloud trial	Useful for a first OAuth2/write spike.	Scope-limited; EAM APIs and write authorisations may not be included.
SAP-provided demo system	Good for a guided proof.	Not your customising; limited lifetime and access.
Local simulator	CI and contract tests.	Not SAP; cannot prove business validation or authorisation.

7.2 Tenant setup checklist

- **Communication system:** register the Django/connector host or user in the SAP communication arrangement.
- **Communication user:** dedicated technical user; no interactive dialog; restrict to the test client and plant.
- **Communication arrangement:** activate the scenario that contains the EAM OData APIs; note the consumer key/secret or OAuth2 client ID/secret and the token endpoint.
- **Business role:** grant the maintenance planner/technician authorisations needed by the test; start read-only, then add create/change.
- **API activation:** confirm each service is active and released; call `$metadata` with a bearer token.
- **Master data:** plant, company code, functional location hierarchy, equipment, planner group, work centre, order type, notification type, material and unit of measure.
- **Test data:** use a dedicated equipment/functional location so test orders are easy to find and clean up.
- **Audit:** agree how test records are cancelled or archived; do not delete records that must remain auditable.

7.3 OAuth2 token

The token endpoint and client credentials come from the communication arrangement. Do not guess the URL or the scope names. Typical response fields are `access_token`, `token_type` and `expires_in`.

```
curl -s -X POST "$TOKEN_URL" \
  -H "Content-Type: application/x-www-form-urlencoded" \
  -d "grant_type=client_credentials" \
  -d "client_id=$CLIENT_ID" \
  -d "client_secret=$CLIENT_SECRET"
```

Cache the token until shortly before expiry; do not request a new token for every OData call. A 401 usually means an expired or invalid token; a 403 means the token is valid but the communication user lacks the authorisation or scope.

7.4 End-to-end write scenario

Step	API / action	Assertion
1. Create notification	POST API_MAINTNOTIFICATION	Notification number returned; equipment, priority and dates match.
2. Read it back	GET the notification by key	Same fields; status and plant are correct.
3. Create order	POST API_MAINTENANCEORDER (or the supported conversion/reference flow)	Order number returned; reference to the notification is preserved.
4. Add operation	POST/PATCH the order operation if the API exposes it	Work centre, planned work and dates match the test plan.
5. Post confirmation	POST API_MAINTENANCEORDERCONFIRMATION	Actual hours and work date visible on the order; final flag set.
6. Change status	PATCH with ETag/If-Match if supported	Conflict (409/412) handled without overwriting a concurrent change.
7. Clean up	Cancel or archive test records	No orphan test data; audit trail retained.

Idempotency is a design decision
SAP does not generally accept a client-generated idempotency key on these APIs. Use a deterministic correlation ID stored in Django, query by your external reference before POST, and treat a timeout as 'unknown' until you reconcile. Retrying a create blindly can duplicate a notification or an order.

7.5 Stage 2 exit criteria

- One end-to-end scenario (notification -> order -> confirmation) passes in the test tenant.
- The communication user can do the required writes and nothing else; read-only and write roles are separated.
- A repeated request does not create duplicates; a 409/412 conflict is handled and surfaced.
- SAP error messages are logged with the HTTP status, SAP code and message text; no secret is logged.
- The test data set is documented and repeatable; cleanup is agreed.

8. Canonical model and field mapping

The demo's canonical model is the right vocabulary for the integration. The connector should map SAP field names into these entities, not the other way round. Field ownership must be explicit: SAP remains the system of record for master data, status, cost and confirmations; the CMMS layer owns permits, RCA, checklists and local notes.

Canonical entity	Key fields	Ownership	Mapping notes
Functional location	code, parent, plant, cost centre, criticality	SAP	Sync parents before children; preserve the hierarchy.
Equipment	asset tag, category, manufacturer, serial, functional location, replacement value	SAP	Criticality may be a custom field; confirm the source.
Notification	number, type, equipment, priority, malfunction start/end, state, text	Shared	Create in CMMS or SAP; decide the system of entry per scenario.
Work order	number, order type, plant, dates, actual cost, state	SAP	Header first; operations and components via navigation.
Operation	operation number, work centre, planned/actual hours	SAP	Confirm the API exposes operations for the order.
Confirmation	technician, hours, work date, final flag	SAP	Post labour; never recompute SAP totals in Django.
Measuring point / reading	meter, reading, unit, reading date	SAP	Separate reading API may be required.
Failure code	problem, cause, remedy, failure mode	SAP	Catalog mapping is often client-specific.

8.1 Mapping rules

- **External ID:** every canonical row stores `(source_system, entity_type, external_id)` with a unique constraint.
- **Status:** map SAP status codes to a small canonical state machine; keep the raw code for audit and support.
- **Dates and times:** store UTC; keep the SAP time zone and the original local value for display and debugging.
- **Units:** normalise to a canonical unit where the dashboard compares values; keep the SAP unit and conversion factor.
- **Currency and precision:** use Decimal, not float; preserve the SAP currency and exchange-rate date.
- **Language:** descriptions may be language-dependent; request the logon language explicitly.
- **Deletes:** prefer a deletion indicator or a full reconciliation; do not infer deletion from absence in one page.
- **Unknown codes:** never drop a row because a status or category is unknown; store it as 'unmapped' and alert.

8.2 Delta and reconciliation

Use an SAP change timestamp where the API exposes one (for example a last-changed field) to request only changed rows. If no reliable delta exists, run a periodic full snapshot for that entity and reconcile by external ID. Keep a high-water mark per entity and connection, and never advance it past a failed page.

Entity	Preferred delta	Fallback	Reconcile
Equipment	Last changed timestamp if exposed	Full snapshot	Count + missing IDs + spot checks
Functional location	Last changed timestamp	Full snapshot	Hierarchy parent/child checks
Notification	Changed-on timestamp	Recent window + full weekly	Open vs closed counts
Order	Changed-on timestamp	Recent window + full weekly	Status and cost totals
Confirmation	Created/changed timestamp	Recent window	Hours by order and work date

9. Django implementation blueprint

The following is a reference shape, not a mandate. Match the existing project's conventions. The important decisions are the external-ID key, the sync-run audit, the checkpoint boundary and the separation between transport (connector) and persistence (Django).

9.1 Suggested app layout

```
integrations/
  sap/
    __init__.py
    client.py          # thin wrapper around erp-connector
    auth.py            # API key vs OAuth2 provider
    mappers.py         # SAP payload -> canonical dicts
    services.py        # sync orchestration, transactions
    tasks.py           # Celery tasks / scheduled jobs
    models.py          # connection, sync run, external id
    admin.py
    tests/
      fixtures/        # recorded sandbox payloads (no secrets)
      test_mappers.py
      test_sync.py
```

9.2 Core models

Model	Purpose	Key fields
ERPConnection	One configured SAP system.	name, system_type, base_url, auth_type, client_id, secret_ref, verify_tls, active
SyncRun	Audit of one entity sync.	connection, entity, mode, started_at, finished_at, status, read/written/failed, cursor, error
ExternalId	Stable link to the SAP key.	source_system, entity_type, canonical_id, external_id (unique together)
FunctionalLocation	Canonical location hierarchy.	code, parent, plant, cost_centre, criticality
Equipment	Canonical asset master.	asset_tag, category, manufacturer, serial, location, replacement_value
MaintenanceNotification	Request or breakdown.	number, type, equipment, priority, malfunction_start/end, state, text
MaintenanceOrder	Execution header.	number, order_type, plant, equipment, dates, actual_cost, state
MaintenanceOperation	Order step.	order, operation_no, work_centre, planned_hours, actual_hours
Confirmation	Labour/time posting.	order, operation, technician, hours, work_date, final
MeasuringPoint	Meter or condition point.	code, equipment, description, unit, last_reading

9.3 Model sketch

```
class ERPConnection(models.Model):
    name = models.CharField(max_length=80, unique=True)
    system_type = models.CharField(max_length=20, default="SAP_S4")
    base_url = models.URLField()
    auth_type = models.CharField(max_length=20)  # api_key | oauth2
    client_id = models.CharField(max_length=200, blank=True)
    secret_ref = models.CharField(max_length=200)  # secret manager key
    verify_tls = models.BooleanField(default=True)
    active = models.BooleanField(default=True)

class SyncRun(models.Model):
    connection = models.ForeignKey(ERPConnection, on_delete=models.PROTECT)
    entity = models.CharField(max_length=40)
    mode = models.CharField(max_length=10)  # delta | full
    status = models.CharField(max_length=20)  # running | ok | failed
    cursor = models.TextField(blank=True)
    read = models.PositiveIntegerField(default=0)
    written = models.PositiveIntegerField(default=0)
    failed = models.PositiveIntegerField(default=0)
    error = models.TextField(blank=True)
    started_at = models.DateTimeField(auto_now_add=True)
    finished_at = models.DateTimeField(null=True, blank=True)
```

Keep the secret out of the database
The `secret_ref` field is a pointer to a secret-manager entry, not the password or client secret itself. A database dump, an admin screenshot or a fixture must never expose a live SAP credential. Rotate the secret without a code change and keep a separate connection per environment.

9.4 Service boundary

```
class SAPSyncService:
    def __init__(self, connection, client):
        self.connection = connection
        self.client = client

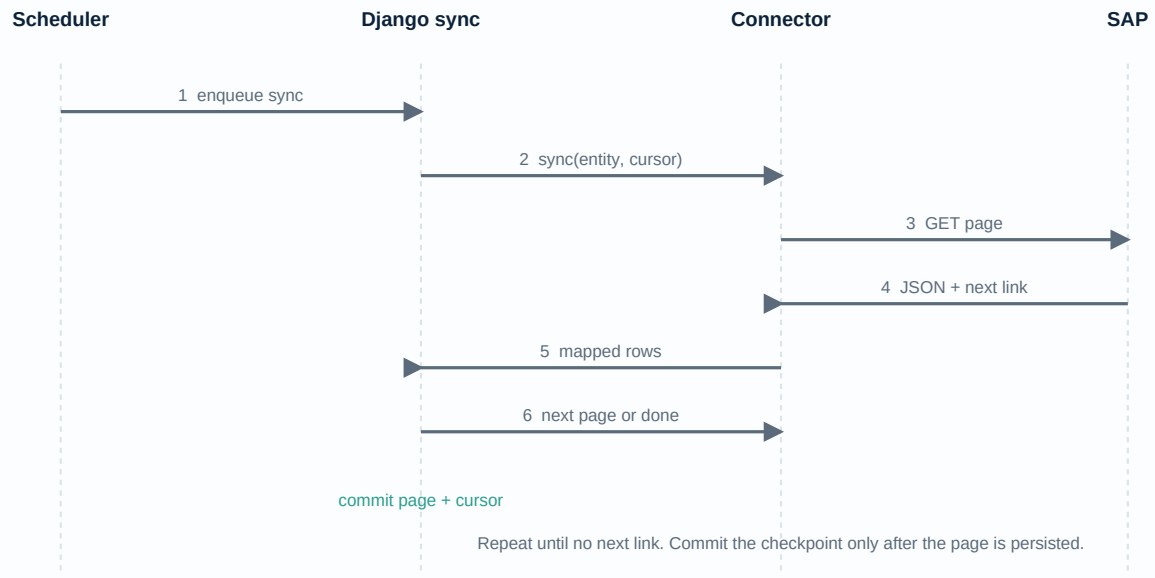
    def sync_equipment(self, run):
        cursor = run.cursor or None
        while True:
            page = self.client.get_page("API_EQUIPMENT", "Equipment", cursor=cursor)
            with transaction.atomic():
                for row in page.rows:
                    self.upsert_equipment(row)
                run.cursor = page.next_cursor or ""
                run.written += len(page.rows)
                run.save(update_fields=["cursor", "written"])
            if not page.next_cursor:
                break
            cursor = page.next_cursor
```

The transaction must include the upserts and the cursor update. If the process dies after the commit, the next run resumes at the saved cursor. If it dies before, the page is replayed and the unique external-ID constraint makes the upserts idempotent.

9.5 Scheduling and operations

- **Celery beat** (or cron + management command) runs one task per entity; a per-connection lock prevents overlapping runs.
- **Retries:** exponential backoff with jitter for 429, 5xx and timeouts; do not retry 400/401/403 blindly.
- **Rate limits:** keep concurrency low, honour `Retry-After`, and separate master-data loads from interactive calls.
- **Admin:** show connection health, last successful run, rows written, errors and a manual 'run now' action.
- **Alerts:** token failures, repeated 429s, a run older than its schedule, and a reconciliation mismatch.
- **Backfill:** a management command that accepts an entity, date range and dry-run flag.

10. Sync, error handling and operations



This is a pull model: SAP does not push changes to Django. The scheduler decides freshness. A common split is every 5-15 minutes for operational notifications and orders, and a nightly full or wide-window pass for master data and reconciliation.

The checkpoint contract
A page is committed only when the upserts and the cursor update share one transaction. If the process dies after the commit, the next run resumes at the saved cursor; if it dies before, the page is replayed and the unique external-ID constraint makes the upserts idempotent. Never advance the cursor first and persist later.

10.1 Error taxonomy

Condition	Typical cause	Action
401 Unauthorized	Expired/invalid token; wrong API key.	Refresh the token; re-check the secret reference; alert after repeated failures.
403 Forbidden	Communication user lacks role/scope; API not active.	Fix the SAP authorisation; do not retry in a loop.
404 Not Found	Wrong service, entity set or key.	Check $metadata; fail the run with a clear message.
400 Bad Request	Invalid filter, date format or payload.	Log the SAP message; fix the mapper; no retry.
409 / 412 Conflict	ETag mismatch or concurrent change.	Re-read the record; merge or surface to a human.
429 Too Many Requests	Rate limit.	Back off, honour Retry-After, reduce concurrency.
500 / 503	SAP or network transient.	Retry with backoff; open a circuit after repeated failures.
Timeout	Network or slow SAP.	Treat create as unknown; reconcile by external reference before retrying.

10.2 Observability

- Log one structured event per page and per failed row: connection, entity, sync run, external ID, HTTP status, SAP code, duration.
- Metrics: rows read/written/failed, page latency, token refresh failures, 429 count, sync lag, last successful run age.
- Dashboards: one panel per connection and entity; red when the last success is older than two schedule intervals.
- Never log tokens, API keys, passwords or full payloads containing personal data.

10.3 Runbook

Situation	First response
Sync is stale	Check the last SyncRun, the scheduler, the token expiry and the SAP connectivity; re-run the entity.
Repeated 403	Confirm the communication user, business role and API activation with SAP security; do not change code first.
Duplicate records	Check the external-ID constraint and the create path; reconcile by external reference; add a regression test.
Token failures	Rotate the secret in the secret manager; redeploy/refresh; confirm the client ID and token URL.
Bad mapping after an SAP update	Re-run the contract tests; compare the new $metadata/payload; update the mapper and fixtures.

11. Security and compliance

SAP integration touches master data, maintenance history and cost. Treat the connector as a privileged system component, not a utility script.

Control	Implementation
Secret storage	Environment variables or a secret manager; the database stores a reference, never the secret. No keys in Git, tickets or screenshots.
TLS	Verify certificates in every environment; never set verify=False. Pin a custom CA only when the tenant requires it.
OAuth2	Client credentials per environment; cache tokens; minimum scopes; separate read-only and write credentials where possible.
Least privilege	Restrict the communication user to the test plant/company code and the required APIs; no dialog user; no SAP_ALL.
Audit	Log who triggered a sync, which connection, which entity, how many rows and which external IDs changed.
PII and retention	Minimise free-text fields; define retention for raw payloads; restrict admin access; encrypt backups.
Network	Allowlist the Django/worker egress to the SAP host; do not expose the connector to the public internet.
Test data	Use synthetic or dedicated test records; never copy production personal data into a demo or a laptop.

The demo stays credential-free
The public Gearbox site is a static export. It must never contain an SAP API key, tenant URL, client secret or live data. The integration belongs in the Django deployment, behind authentication and a server-side secret store.

12. Test strategy and acceptance criteria

Level	Environment	What is tested	Pass criteria
Unit	Local / CI	Mappers, status maps, unit conversions, retry logic, cursor handling.	Deterministic; no network; coverage of edge cases.
Contract	Simulator / fixtures	Connector request shape, pagination, V2/V4 parsing, error normalisation.	Green on every connector change; fixtures refreshed from sandbox.
Sandbox	Business Accelerator Hub	GET metadata and pages; filters; paging; error codes.	Four APIs read; mapping notes signed off; no secret committed.
Tenant	S/4HANA test tenant	OAuth2, authorisation, create notification/order, confirmation, conflict.	E2E scenario passes; no duplicates; cleanup done.
UAT	Tenant + Django UI	Breakdown -> notification -> order -> confirmation; permissions; reporting.	Maintenance SME accepts; known issues logged.
Non-functional	Tenant / staging	Volume, latency, rate limits, recovery, token expiry.	Within agreed budget; retries and alerts work.

12.1 UAT scenarios

- A technician reports a breakdown against an equipment; a notification is created with the right priority and malfunction start.
- The planner converts or links the notification to a work order; the order shows the correct plant, order type and dates.
- The technician posts hours against the operation; the confirmation appears on the order with the right work date.
- A second user changes the order while the first is editing; the conflict is detected, not overwritten.
- SAP is unavailable during a scheduled sync; the run fails cleanly, alerts, and resumes without duplicates.
- A user without the write role attempts a create; the app shows a clear permission error and does not retry.

12.2 Definition of done

The integration is done for the pilot when one read path and one write path are repeatable in a test tenant, the canonical data is visible in Django, the runbook is exercised, secrets are rotated, and the maintenance owner signs off the field mapping. Production readiness also needs change approval, monitoring, backup/restore and a rollback plan.

13. Delivery plan and responsibilities

Phase	Duration	Owner	Output
0. Access and decisions	1-2 days	Product + engineering	Connector/Django access; SAP account; tenant request; API list.
1. Sandbox read-only	3-5 days	Django developer	Smoke tests; paging; mapping notes; error taxonomy.
2. Django persistence	5-8 days	Django developer	Models, migrations, services, tasks, admin, tests.
3. Tenant write path	5-10 days	SAP basis + engineering	OAuth2, communication arrangement, E2E notification/order/confirmation.
4. Hardening	3-5 days	Engineering + ops	Retries, reconciliation, monitoring, secrets, runbook.
5. UAT and sign-off	3-5 days	Maintenance SME + QA	Accepted scenarios; known issues; go/no-go.

13.1 Who does what

- **Product/engineering lead:** scope, API list, acceptance criteria, access escalations.
- **Django developer:** models, services, tasks, admin, tests, deployment.
- **Connector owner:** supported APIs, OData versions, auth providers, simulator, contract tests.
- **SAP basis/security:** communication system/user/arrangement, roles, scopes, API activation, transport.
- **Maintenance SME:** field mapping, order/notification types, UAT scenarios, sign-off.
- **QA:** test plan, regression, non-functional checks, evidence.

13.2 First 10 working days

Day	Action
1	Create the Hub account; copy the API key; run the metadata smoke test.
2	Query one page from each of the four APIs; record entity sets and field names.
3	Map 10 equipment and 5 locations; list unmapped fields and owner decisions.
4	Map notifications and orders; write the error taxonomy; run paging tests.
5	Stage 1 review; freeze the canonical field list; request the test tenant if not already open.
6-8	Add Django models and the sync service for equipment and locations; fixtures and unit tests.
9-10	Add notifications and orders read sync; admin visibility; checkpoint and retry tests.

14. Risks, open questions and next steps

Risk / question	Impact	Mitigation
erp-connector code is not accessible	Cannot confirm API coverage or write support.	Provide a local path/public access; review before committing to dates.
No Django app in this workspace	Integration shape is an assumption.	Share the Django repo; adapt models, tasks and settings.
Public sandbox is read-only	Writes cannot be proven for free.	Book a test tenant early; use the simulator only for CI.
EAM APIs not in the trial/role	Stage 2 blocks.	Confirm the communication scenario and scopes with SAP basis before setup.
Test master data missing	Orders/notifications fail validation.	Prepare plant, equipment, work centre, order/notification types and planner group.
Mapping ambiguity	Wrong KPIs or status.	Field ownership workshop; keep raw codes; sign off with the maintenance SME.
Rate limits / volume	Slow or failed sync.	Page size, concurrency, backoff, off-peak windows, monitoring.
Secret leakage	Severe security incident.	Secret manager, least privilege, rotation, no secrets in logs or the static demo.
Scope creep to ECC/Maximo/Oracle	Pilot slips.	Keep the pilot on S/4HANA; the other adapters remain planned.

14.1 Open questions to answer before build

- Is `erp-connector` a Python package with a stable public API, or a set of scripts? Which Python versions does it support?
- Which SAP APIs and OData versions are implemented? Are writes implemented for notifications, orders and confirmations?
- How does the connector handle OAuth2 token refresh, ETags, SAP error payloads and paging?
- Which Django version, database and task runner are in use? Is there an existing integration app?
- Which S/4HANA edition and tenant will be used for Stage 2? Are the EAM APIs in the licensed scope?
- Who owns the communication arrangement, business role and test master data?
- What is the write scope for the pilot: notification only, or notification -> order -> confirmation?
- What are the data residency, retention and audit requirements?

14.2 Recommended next step

> **Do this first**
> Provide access to **erp-connector** and the **Django application**, and create the SAP Business Accelerator Hub account. With those three inputs, the connector review and the Stage 1 smoke test can run in parallel. The tenant request should be raised on the same day, because it is the long pole for write testing.

Appendix A. API inventory and request cookbook

Use this as a field checklist, not as a substitute for the service metadata. The exact entity sets, properties and navigation paths are defined by each API's EDMX document.

Service	Business object	Read	Create	Change	Notes
API_EQUIPMENT	Equipment master	Yes	Check	Check	Master data; often read-only in a CMMS integration.
API_FUNCTIONALLOCATION	Functional location	Yes	Check	Check	Hierarchy; sync parents first.
API_MAINTNOTIFICATION	Maintenance notification	Yes	Yes	Yes	Breakdown/request; status actions may vary.
API_MAINTENANCEORDER	Maintenance order	Yes	Yes	Yes	Header, operations, components; confirm navigation.
API_MAINTENANCEORDERCONFIRMATION	Confirmation	Yes	Yes	Check	Labour/time; final confirmation flag.
API_MEASURINGPOINT	Measuring point	Yes	Check	Check	Readings may use a separate API.

A.1 OData query cheat sheet

Option	Use	Example
$top	Page size	?$top=50
$skip	Offset paging (not all services)	?$skip=100
$select	Limit properties	?$select=Equipment,EquipmentName
$filter	Server-side filter	?$filter=Plant eq '1000'
$orderby	Stable ordering for paging	?$orderby=Equipment
$expand	Include navigation	?$expand=to_Equipment
$count	Total count (where supported)	?$count=true
$skiptoken	Server cursor from the next link	?$skiptoken=...

A.2 Minimal requests

```
# Metadata
curl -s -H "APIKey: $SAP_API_KEY" \
  "$BASE/API_EQUIPMENT/\$metadata" -o equipment.edmx

# One page, selected fields
curl -s -H "APIKey: $SAP_API_KEY" -H "Accept: application/json" \
  "$BASE/API_EQUIPMENT/Equipment?\$top=10&\$select=Equipment,EquipmentName"

# Tenant: bearer token instead of APIKey
curl -s -H "Authorization: Bearer $ACCESS_TOKEN" -H "Accept: application/json" \
  "https://<tenant-host>/sap/opu/odata/sap/API_EQUIPMENT/Equipment?\$top=10"
```

A.3 HTTP status handling

Status	Meaning	Retry?
200	Success	No
201	Created	No; store the returned key
204	No content	No
400	Bad request / validation	No; fix the request
401	Unauthenticated	Refresh token once, then alert
403	Forbidden / missing role	No; fix SAP authorisation
404	Wrong service/entity/key	No; fix the path
409 / 412	Conflict / ETag mismatch	Re-read and reconcile
429	Rate limited	Yes, with Retry-After/backoff
500 / 503	Server/transient	Yes, with capped backoff

OData V2 responses usually wrap the payload in `d.results` and the next link in `d.__next`. V4 uses `value` and `@odata.nextLink`. Confirm which shape each service returns and normalise it once, in the connector.

Appendix B. Glossary and references

Term	Meaning
OData	Open Data Protocol; the REST-based query protocol used by the SAP EAM APIs.
EAM / PM	Enterprise Asset Management / Plant Maintenance.
Functional location	The physical or logical location where maintenance work happens.
Equipment	The asset or machine being maintained.
Notification	A request, malfunction or breakdown record.
Maintenance order	The execution document for the work, with operations and costs.
Confirmation	The record of labour, time and work performed.
Measuring point	A meter or condition point with readings.
Communication arrangement	SAP configuration that binds a communication system, user, scenario and scopes.
Communication user	Technical user used by an external system; no interactive login.
Scope	The permission granted to an OAuth2 client by the communication arrangement.
ETag / If-Match	Optimistic concurrency control for updates.
Idempotency	The property that repeating the same request does not create a duplicate effect.
Delta sync	Fetching only records changed since the last successful cursor.

B.1 References

- SAP Business Accelerator Hub — https://api.sap.com/ (API catalogue, sandbox and API key).
- SAP Help Portal — https://help.sap.com/ (communication arrangements, communication users, OAuth2 and EAM APIs).
- OData — https://www.odata.org/documentation/ (protocol and query options).
- Django — https://docs.djangoproject.com/ (models, transactions, settings, testing).
- Celery — https://docs.celeryq.dev/ (tasks, beat, retries).
- GitHub Pages custom domains — https://docs.github.com/en/pages/configuring-a-custom-domain-for-your-github-pages-site
- Gearbox demo — https://gearbox.nirmanaerp.com/ and https://github.com/BusDec/cmms-demo

B.2 Readiness checklist

Check	Evidence to attach
[ ] Connector interface confirmed	Supported APIs, OData versions, auth providers and write coverage.
[ ] Django models migrated	ERPConnection, SyncRun, ExternalId and canonical tables in staging.
[ ] Sandbox read path green	Metadata and one page from each of the four EAM APIs.
[ ] Mapping signed off	Field map with owner, transform, unit and unmapped-field decision.
[ ] Tenant and arrangement ready	Communication system, user, arrangement, role and scopes.
[ ] OAuth2 token proven	Token request, expiry handling and 401 recovery.
[ ] Write E2E proven	Notification -> order -> confirmation in the test tenant.
[ ] No duplicates	Replay and timeout tests show idempotent creates/updates.
[ ] Reconciliation running	Daily count/checksum and a resolution path for mismatches.
[ ] Monitoring and alerts	Sync lag, failures, 429s, token errors and a runbook owner.
[ ] Secrets rotated	Secret-manager references, rotation and no keys in Git or logs.
[ ] UAT sign-off	Maintenance SME acceptance and a known-issues list.

Document status: architecture-level assessment, version 1.0. The connector and Django source were not available for code review. Update the connector-specific sections after access is granted. No client data, credentials or production records are included. Gearbox is an interim demo name and is not SAP-certified.